

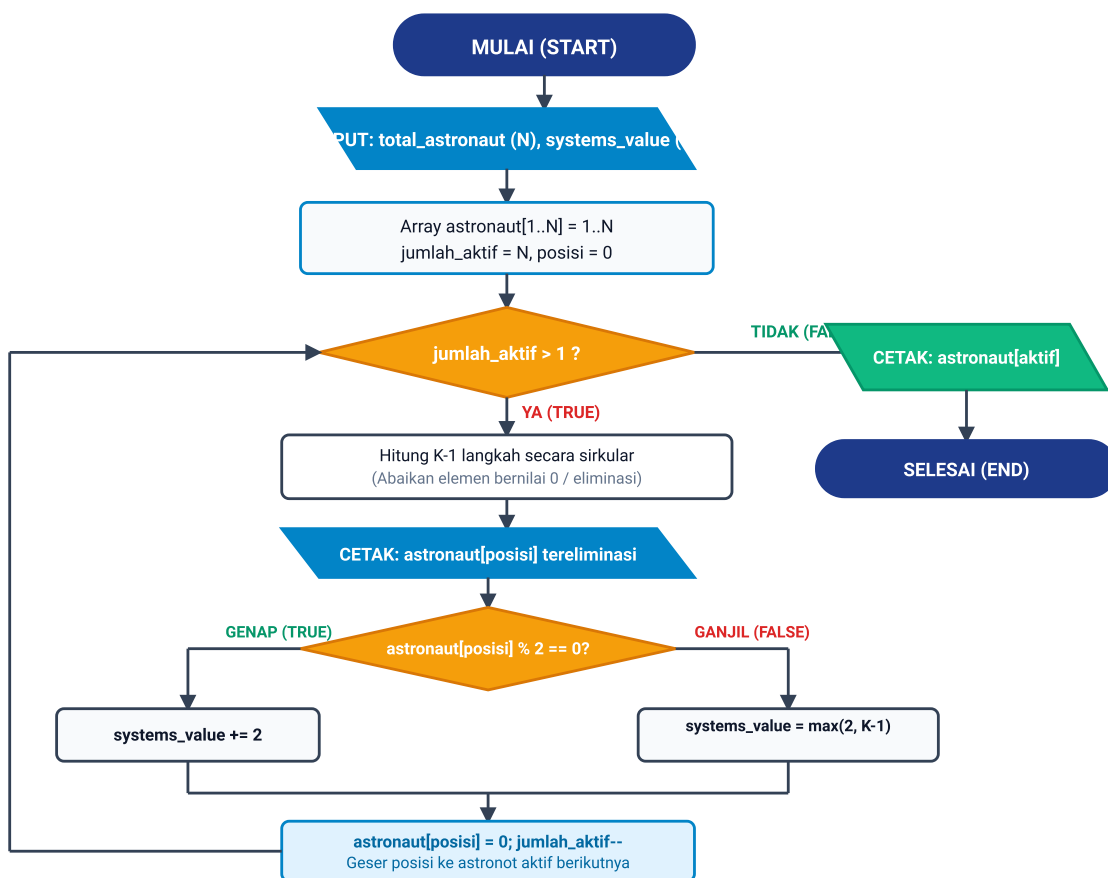
PROBLEM 1: THE LAST ASTRONAUT

FLOWCHART SYSTEM

Deskripsi Singkat Permasalahan:

Sebanyak N astronot membentuk lingkaran dan dieliminasi secara bertahap berdasarkan kelipatan nilai sistem K. Setiap astronot tereliminasi, nilai K berubah secara dinamis: bertambah +2 jika ID bernilai genap, dan berkurang -1 (minimum 2) jika ID bernilai ganjil. Proses berlanjut hingga tersisa 1 astronot.

Flowchart Algoritma Simulasi Eliminasi



1. Konsep Dasar & Variasi Josephus Problem

Masalah **The Last Astronaut** merupakan pemodelan matematis dari permasalahan klasik *Josephus Problem*. Pada bentuk klasiknya, langkah eliminations bernilai konstan. Namun pada soal ini, digunakan aturan **dinamis berbasis paritas (ganjil/genap)** dari ID elemen yang tereliminasi. Permasalahan ini membutuhkan koordinasi struktur data linier yang dikelola secara sirkular.

2. Komponen Utama & Struktur Data

Representasi Lingkaran

Digunakan array `astronaut[1..N]` berukuran `N`. Penandaan elemen tereliminasi dilakukan dengan mengubah nilainya menjadi 0. Elemen bernilai 0 tidak lagi dihitung pada perputaran berikutnya.

Pengendali Status (State Variables)

- `jumlah_aktif`: Menelusuri sisa astronot.
- `posisi`: Indeks pointer penunjuk astronot saat ini.
- `systems_value` (`K`): Jumlah langkah eliminasi.

3. Mekanisme Penelusuran Sirkular & Penyesuaian Indeks

Algoritma melakukan pencarian lokasi astronot yang akan dieliminasi dengan melangkah sebanyak `K-1` kali dari posisi saat ini. Setiap perpindahan indeks menggunakan operasi aritmatika modular:

$$\text{posisi} = (\text{posisi} + 1) \text{ MOD } \text{total_astronaut}$$

Selama pergeseran, jika algoritma menemukan elemen bernilai 0 (astronot yang sudah tereliminasi), penunjuk akan terus bergeser tanpa mengurangi kuota langkah `K`.

4. Aturan Pembaruan Dinamis Nilai K (Systems Value)

Setelah menentukan astronot yang gugur pada posisi tertentu, algoritma mengevaluasi paritas nomor ID astronot tersebut (`astronaut[posisi]`):

- **Apabila ID Genap** (`ID % 2 == 0`): Sistem mengalami akselerasi. Nilai langkah ditambah 2 (`K = K + 2`). Hal ini mempercepat jarak eliminasi pada putaran berikutnya.
- **Apabila ID Ganjil** (`ID % 2 != 0`): Sistem mengalami pembatasan. Nilai langkah dikurangi 1 (`K = K - 1`).
- **Batas Minimum Safety**: Jika pengurangan menyebabkan `K < 2`, maka nilai `K` dikembalikan secara otomatis menjadi 2 untuk menjaga agar langkah eliminasi tetap valid dan tidak bernilai nol atau negatif.

5. Contoh Penelusuran Eksekusi (Trace Table Singkat N=5, K=3)

Putaran	Astronot Aktif	Nilai K	ID Tereliminasi	Paritas ID	Nilai K Baru
1	[1, 2, 3, 4, 5]	3	Astronot 3	Ganjil	$3 - 1 = 2$
2	[1, 2, 0, 4, 5]	2	Astronot 5	Ganjil	$\max(2, 2 - 1) = 2$
3	[1, 2, 0, 4, 0]	2	Astronot 2	Genap	$2 + 2 = 4$
4	[1, 0, 0, 4, 0]	4	Astronot 4	Genap	$4 + 2 = 6$
Hasil	Astronot 1 adalah kandidat terakhir yang bertahan (The Last Astronaut).				

Pseudocode Program Dekripsi Teks

```
PROGRAM Alien_In_The_Middle DEKLARASI encrypted : ARRAY [1..100] OF CHAR i, sebelumnya, s
```

Penjelasan Logika & Mekanisme Kriptografi

Problem **Alien In The Middle** menerapkan teknik dekripsi berbasis *Sequential Difference Cipher*. Teks terenkripsi dibaca karakter demi karakter dan ditransformasikan kembali ke karakter aslinya berdasarkan urutan posisi alfabet (\$1\$ hingga \$26\$).

1. Formulasi Matematis Dekripsi

Misalkan C_i adalah nilai urutan alfabet dari karakter terenkripsi ke-\$i\$, dan P_i adalah nilai alfabet karakter asli (plaintext) ke-\$i\$:

- Untuk $i = 1$: Karakter pertama tidak tergeser, sehingga $P_1 = C_1$.
- Untuk $i > 1$: Nilai asli diperoleh dengan formula pengurangan berantai:

$$P_i = (C_i - P_{i-1}) \bmod{26}$$

2. Penanganan Underflow / Nilai Negatif (+26 Offset)

Dalam representasi komputer, operasi sekarang - sebelumnya dapat menghasilkan nilai kurang dari atau sama dengan nol (\$0\$ atau negatif). Kondisi ini diatasi dengan menambahkan konstanta \$26\$ (jumlah total abjad alfabet). Hal ini menjamin bahwa hasil pemetaan selalu berada dalam domain rentang terdefinisi \$[1, 26]\$.

3. Contoh Penelusuran Dekripsi (Trace Table)

Simulasi dekripsi untuk masukan teks terenkripsi: "KAZ"

i	Karakter (C)	Nilai C	sebelumnya (P_{i-1})	Kalkulasi ($C - P$)	Hasil (+26)	Plaintext (P)
0	'K'	11	-	Langsung	11	'K'
1	'A'	1	11 ('K')	$1 - 11 = -10$	$-10 + 26 = 16$	'P' (urutan 16)
2	'Z'	26	16 ('P')	$26 - 16 = 10$	10	'J' (urutan 10)
Hasil Dekripsi Akhir			Teks asli terdekripsi: KPJ			